The Journal of Open Source Software

VibIR-Parallel-Compute: Enhancing Vibration and Infrared Analysis in High-Performance Computing Environments

Kurt Irvin M. Rojas [1¶], **Yoshitada Morikawa** [1,2], **and Ikutaro Hamada** [1]

1 Department of Precision Engineering, Graduate School of Engineering, Osaka University, 2-1, Yamadaoka, Suita, Osaka, 565-0871, Japan **2** Research Center for Precision Engineering, Graduate School of Engineering, Osaka University, 2-1, Yamadaoka, Suita, Osaka, 565-0871, Japan ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 02 November 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

In this work, we introduce `VibIR-Parallel-Compute`, a Python package designed to enhance both the usability and efficiency of vibrational and infrared (IR) analysis, particularly for high-performance computing environments. To address the computational bottleneck encountered in the vibrational spectroscopy modules in `Atomic Simulation Environment` (ASE) (Hjorth Larsen et al., 2017), we provide a *drop-in* replacement that parallelizes these bottleneck calculations in the current implementation. Additionally, we offer customizable workflows for processing the data required for vibrational and IR analysis, enabling previously incompatible calculators to now be compatible. Furthermore, our package includes streamlined and efficient analysis modules that allow for quick visualization of vibrational modes, along with the plotting of vibrational density of states (VDOS) and IR spectra, all without the need to explore the underlying source code or manage excessive boilerplate.

Statement of need

Vibrational spectroscopy has become a valuable technique for analyzing structures of molecules and materials (Kraka et al., 2020). IR spectroscopy, in particular, is widely utilized to examine the inherent strength of chemical bonds and the characteristic bonding patterns present in materials. However, assigning vibrational modes to experimental spectra is challenging and typically requires comparison with model spectra predicted through electronic structure calculations, such as density functional theory (DFT).

A commonly used and efficient method for determining vibrational characteristics, such as modes and frequencies, involves solving for the eigenvector and eigenvalues of a Hessian matrix (Giustino, 2014; Wilson et al., 1980). This matrix represents the strength of chemical bonds under the harmonic approximation. Each element in the Hessian matrix is defined as:

$$H_{ij} = -\frac{\partial F_j}{\partial u_i} \tag{1}$$

where the Hessian element H_{ij} reflects the negative gradient of the force F_j with respect to atomic position u_i. The i and j span the three spatial coordinates across N atoms.

The force gradient is determined using the finite-difference method, which involves applying small forward and backward displacements to atoms. This requires calculating the forces twice – once for the forward displacement and once for the backward displacement – both of

which are independent computations. This independence is crucial as it enables parallel force evaluations for different displaced structure.

The construction and diagonaliation of the Hessian matrix are implemented in the widely-used Python package, ASE. ASE offers various methods and workflow that interface with several *ab initio* calculators, such as VASP (Kresse & Hafner, 1993), Quantum Espresso (Giannozzi et al., 2009), and GPAW (Mortensen et al., 2024) . Modules related to spectroscopy, specifically, can be found in the `ase.vibrations` module. In ASE's implementation, the total number force evaluations N_{calc} required by a system with N_{atom} atom is given by:

$$N_{\mathrm{calc}} = 6 \cdot N_{\mathrm{atom}} + 1 \tag{2}$$

In this equation, the factor of 6 accounts for the three spatial directions, considering both forward and backward displacements. The additional $+1$ term is for the equilibrium structure. This total number of calculations ensures compatibility with all three finite-difference methods: central, forward, and backward.

Although parallelizing the calculations for the displaced systems is straightforward, the current implementation runs in a serial loop with a time complexity of $\mathcal{O}(N_{\mathrm{calc}})$. This limits the ability to take full advantage of the large computing resources typically available in high-performance computing (HPC) environments. While parts of the existing code could be adapted (along with new compatibility code) to implement parallelization, doing so would increase complexity and reduce the clarity of the script-level codes. Therefore, a more user-friendly, parallelized version of the `ase.vibrations` toolkit is desirable.

Another issue arises from the need for an additional calculation input and output processing layer. When generating IR spectra, we require the dipole moment changes of the system. In a three-dimensional system, such as an isolated molecule, this involves calculating a three-dimensional dipole moment vector. Unfortunately, some calculators – such as Quantum Espresso – only computes and outputs the dipole moment as a scalar along the chosen electric field direction (typically along the x, y, and z axis). This means that three separate calculations are necessary to obtain the full dipole moment vector. Currently, ASE does not support adding a processing layer capable of handle these complex but necessary workflows.

Features and Implementation

Parallel ready code

In this work, we focused on enhancing the existing code to support parallelization by distributing each independent atomically displaced atomic system across separate computing resources. This approach, in theory, allows all systems to run simultaneously. We leveraged resource workload managers commonly found in high-performance computing environments (e.g. PBS, SLURM, SGE) to efficiently assign each system to a distinct resource pool.

DRAFT

The Journal of Open Source Software

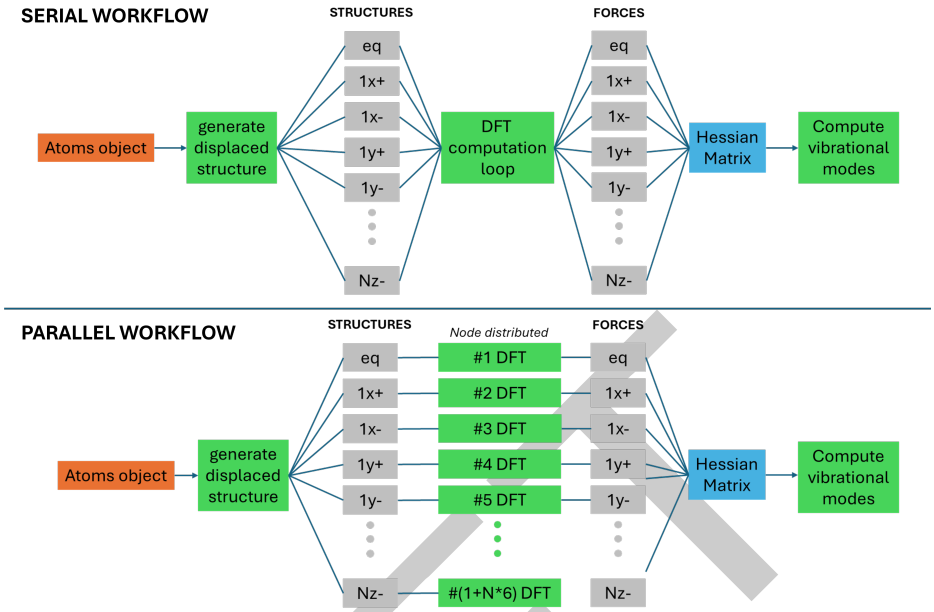

Figure 1: Workflow comparison. The serial and parallel workflow are compared, with the key difference being how resources are allocated for each displaced structure. In the structure/forces section, "eq" refers to the equilibrium structure, while "$1x+$" denotes atom 1 forward displced along x-axis.

Figure 1 compares the workflows of the serial and parallel implementations. In the serial version, all structures are assigned to a single resource pool, while in the parallel version, each structure is distributed to a separate pool, enabling simultaneous calculations. The serial loop relies on the `run()` method, but we modified the code to support parallelization by adopting a module approach. We introduced a `run_tag(i)` method to run individual systems based on their index which can be executed using job arrays in resource workload managers. The following is a pseudo code for the difference between the serial and parallelized version:

```
# serial
vib = Vibrations(atoms)
vib.run()

# parallel - in serial loop
pvib = ParallelVibrations(atoms, calc_obj=EMT)
for i in range(1, vib.map_length +1):
   pvib.run_tag(i)

# parallel - parallelized
pvib = ParallelVibrations(atoms, calc_obj=EMT)
pvib.run_tag(int(os.environ['PBS_ARRAY_INDEX']))
```

Timings

The timing results in Figure 2 illustrate the distinct complexities associated with each implementation scenario. In the serial implementation, the time complexity is $\mathcal{O}(N_{\text{calc}})$. By contrast, in a fully parallelized version – where all needed resources are available – it reduces to $\mathcal{O}(1)$. However, in practice, this ideal scenario is rarely achieved due to HPC fair-use policies that limit the number of resources a user can access simultaneously. This limited-resource strategy can be referred to as *batch parallel*, where jobs are grouped into resource-constrained batches to meet fair-use requirements. The third case in the figure illustrates a batch parallel scenario with a fair-use limit of 10 simultaneous jobs, leading to an effective time complexity

DRAFT

of $\mathcal{O}(\lceil N_{\text{calc}}/10 \rceil)$.

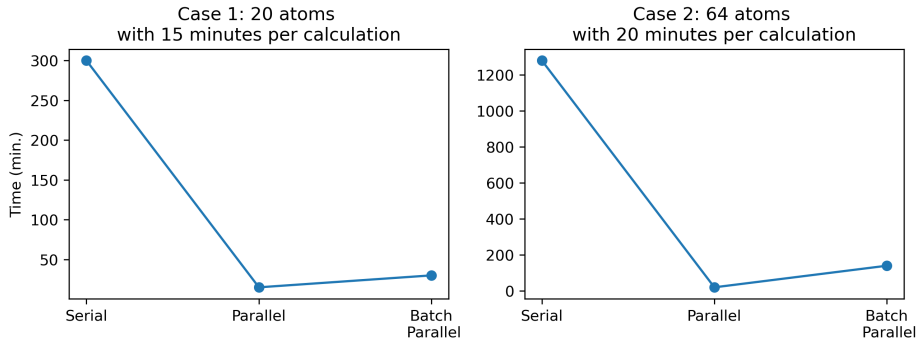

Figure 2: Example timings. We present two cases to demonstrate how parallelization reduces overall real-time spent. Additionally, a batch parallel run is considered to highlight the impact of fair-use policies on parallelization timings.

Temporary files issue

One downside of this parallelized approach is the generation of numerous temporary files from the calculations, which can be quite large (e.g. charge density, wavefunction data). On larger systems, this could risk exhausting disk space. To address this, we implemented a file management system that allows deletion of temporary files after the calculations are complete. Additionally, since multiple calculations run in parallel, we created a system that ensures each calculation operates in a separate working directory, preventing any accidental sharing or overwriting of output and temporary files.

Processing layer

In this additional processing layer, we designed the calculation to be flexible, allowing for more complex workflows. Our focus was on the applicability of Quantum Espresso for infrared spectra calculations. As mentioned earlier, Quantum Espresso only calculates the dipole moment in a single direction. However, for molecular, ribbon, and sheet systems, we require three-dimensional, two-dimension, and one-dimensional dipole moment vectors, respectively. To handle this, we implemented a three-step calculation for each dipole direction and constructed the full dipole moment vector from these results. Additionally, for ribbon and sheet sytems, we skip calculating the dipole moment along periodic directions to save time and resources. A schematic of the processing layer is shown in Figure 3.

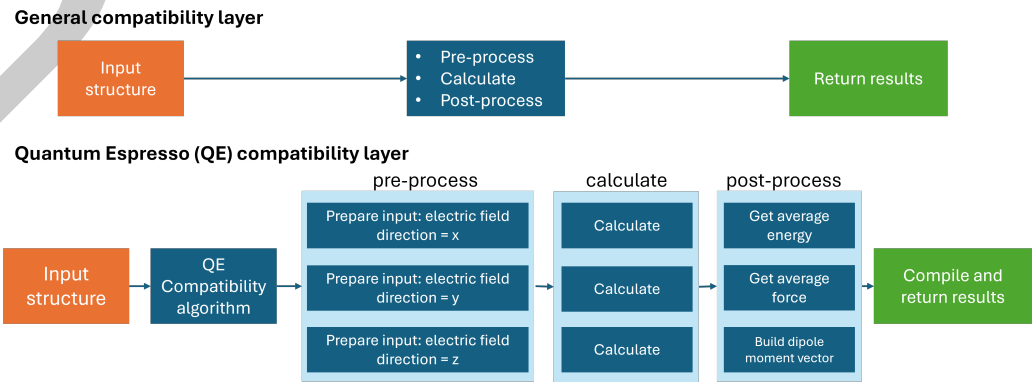

Figure 3: Compatability layer. We present the diagram of the general compatability layer and a specific example of the Quantum Espresso compatability layer.

DRAFT

Fast analysis

While ASE provides tools for analyzing vibrational modes and frequencies, we enhanced its user-friendliness. We introduced a quick-plot feature for vibrational modes and frequencies in both Vibrations (Figure 4) and Infrared (Figure 5)analyses. Additionally, we implemented a more detailed GIF animation of the vibrational modes to offer a clearer, more intuitive understanding. To improve efficiency, we applied CPU-based parallelization for GIF generation, significantly speeding up the process.

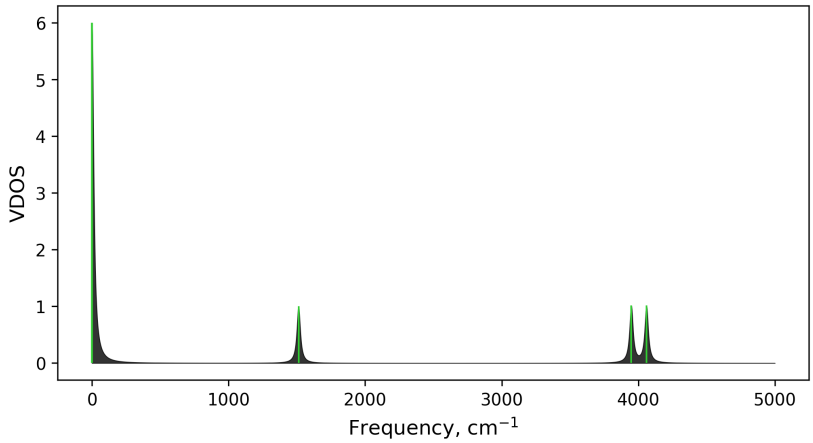

Figure 4: Vibrational density of states of water.

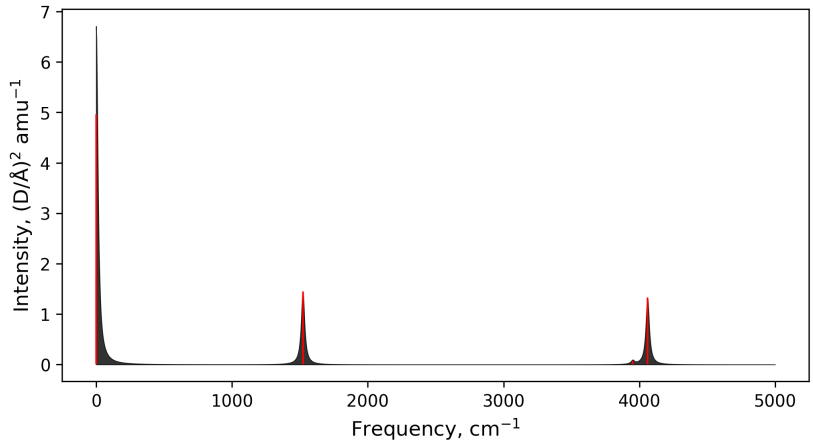

Figure 5: Simulated infrared spectra of water.

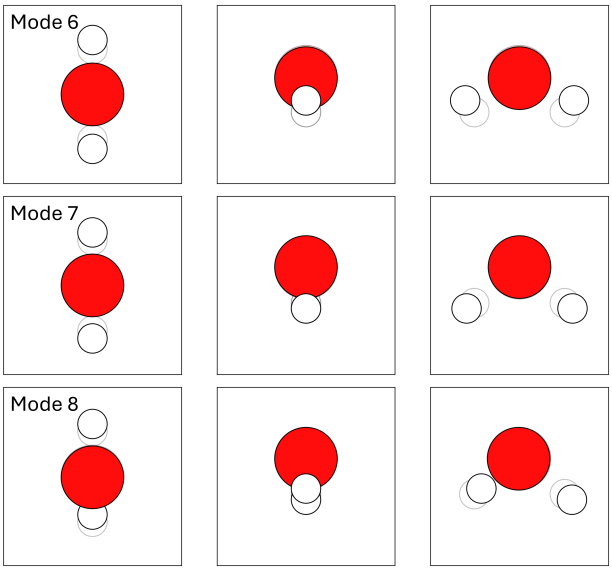

Figure 6: Static representation of mode GIFs. The image includes mode 6, 7 and 8 of water as an example.

Usage

The `VibIR-Parallel-Compute` package is available on Gitlab https://gitlab.com/morikawa-lab-osakau/vibir-parallel-compute and documentations are provided in https://morikawa-lab-osakau.gitlab.io/vibir-parallel-compute.

References

Giannozzi, P., Baroni, S., Bonini, N., Calandra, M., Car, R., Cavazzoni, C., Ceresoli, D., Chiarotti, G. L., Cococcioni, M., Dabo, I., Dal Corso, A., De Gironcoli, S., Fabris, S., Fratesi, G., Gebauer, R., Gerstmann, U., Gougoussis, C., Kokalj, A., Lazzeri, M., … Wentzcovitch, R. M. (2009). QUANTUM ESPRESSO: A modular and open-source software project for quantum simulations of materials. *Journal of Physics: Condensed Matter*, *21*(39), 395502. https://doi.org/10.1088/0953-8984/21/39/395502

Giustino, F. (2014). *Materials modelling using density functional theory: Properties and predictions* (1st ed). Oxford University Press. ISBN: 978-0-19-966244-9

Hjorth Larsen, A., Jørgen Mortensen, J., Blomqvist, J., Castelli, I. E., Christensen, R., Dułak, M., Friis, J., Groves, M. N., Hammer, B., Hargus, C., Hermes, E. D., Jennings, P. C., Bjerre Jensen, P., Kermode, J., Kitchin, J. R., Leonhard Kolsbjerg, E., Kubal, J., Kaasbjerg, K., Lysgaard, S., … Jacobsen, K. W. (2017). The atomic simulation environment—a Python library for working with atoms. *Journal of Physics: Condensed Matter*, *29*(27), 273002. https://doi.org/10.1088/1361-648x/aa680e

Kraka, E., Zou, W., & Tao, Y. (2020). Decoding chemical information from vibrational spectroscopy data: Local vibrational mode theory. *WIREs Computational Molecular Science*, *10*(5), e1480. https://doi.org/10.1002/wcms.1480

Kresse, G., & Hafner, J. (1993). *Ab Initio* molecular dynamics for liquid metals. *Physical Review B*, *47*(1), 558–561. https://doi.org/10.1103/PhysRevB.47.558

Mortensen, J. J., Larsen, A. H., Kuisma, M., Ivanov, A. V., Taghizadeh, A., Peterson, A., Haldar, A., Dohn, A. O., Schäfer, C., Jónsson, E. Ö., Hermes, E. D., Nilsson, F. A., Kastlunger, G.,

DRAFT

Levi, G., Jónsson, H., Häkkinen, H., Fojt, J., Kangsabanik, J., Sødequist, J., … Thygesen, K. S. (2024). GPAW: An open Python package for electronic structure calculations. *The Journal of Chemical Physics*, *160*(9), 092503. https://doi.org/10.1063/5.0182685

Wilson, E. B., Decius, J. C., & Cross, P. C. (1980). *Molecular vibrations: The theory of infrared and raman vibrational spectra* (First published in 1980, is an unabridged and corrected republication of the work originally published in 1955by McGraw-Hill Book Company, Inc). Dover Publications Inc. ISBN: 978-0-486-63941-3

DRAFT